

LAB # 08: HUFFMAN ENCODING

Lab Objective:

Compress and decompress text file using Huffman encoding-decoding algorithm.

Lab Description:

Huffman coding is a lossless data compression algorithm. The idea is to assign variable-length codes to input characters, lengths of the assigned codes are based on the frequencies of corresponding characters. The principle is to use a lower number of bits to encode the data that occurs more frequently. Codes are stored in a Code Book which may be constructed for each image or a set of images. In all cases the code book plus encoded data must be transmitted to enable decoding.

Algorithm:

1. Create a leaf node for each unique character and build a min heap of all leaf nodes.
2. Extract two nodes with the minimum frequency from the min heap.
3. Create a new internal node with a frequency equal to the sum of the two nodes frequencies. Make the first extracted node as its left child and the other extracted node as its right child. Add this node to the min heap.
4. Repeat steps#2 and #3 until the heap contains only one node. The remaining node is the root node and the tree is complete.

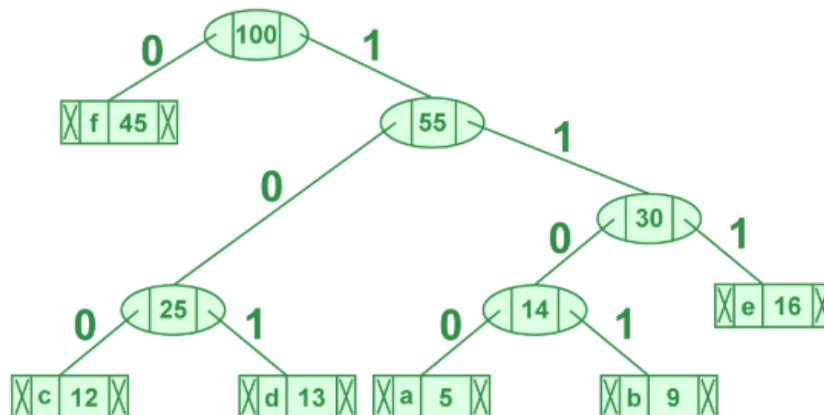


Figure 13. 1

Lab Task:

1. Design and implement a **file compression and decompression system** using **Huffman Encoding in C++**.

The program must:

- Read text data from an input file
- Generate Huffman codes based on character frequencies
- Encode and store the compressed data in an output file
- Read the compressed file and reconstruct the original text
- Write the decompressed output to a new file

The system should ensure **lossless compression**, meaning the decompressed file must exactly match the original input.

THINK:

1. How does the frequency of characters affect the structure of the Huffman tree and the resulting encoded message?
2. Why might Huffman encoding fail to significantly compress certain types of data? Can you give examples?